

Bug Identified

Bug 1: SQL Operator Precedence Bug

Location: backend/src/main/java/com/internal/tasktracker/TaskRepository.java (Line 14-17), db/queries/search_tasks.sql, db/oracle/task_search_package.sql

How Discovered: Code review of the @Query annotation and checking SQL behavior when searching by description.

Root Cause: In SQL, AND has higher precedence than OR. Without parentheses around (LOWER(title) LIKE ... OR LOWER(description) LIKE ...), the query was evaluated as (archived = FALSE AND title) OR (description AND status). This caused archived tasks to leak when description matched, and bypassed the status filter when title matched.

Fix & Rationale: Wrapped the title and description LIKE conditions inside parentheses. This ensures archived = FALSE and status filtering are strictly applied across both search fields.

Bug 2: Artificial Request Latency

Location: backend/src/main/java/com/internal/tasktracker/TaskController.java (Lines 36-42)

How Discovered: Reviewing backend performance logic in searchTasks endpoint.

Root Cause: An artificial weight calculation Thread.sleep(Math.max(0, 10 - query.length()) * 100L) caused a forced 1-second blocking sleep on empty search queries.

Fix & Rationale: Removed the Thread.sleep and complexity logic completely so search responses return instantly without blocking request threads.

Bug 3: Unhandled Exception on Status Enum Parsing

Location: backend/src/main/java/com/internal/tasktracker/TaskController.java (Lines 31-33)

How Discovered: Testing status parameter inputs in TaskController.

Root Cause: TaskStatus.valueOf(status.toUpperCase()) throws an unhandled IllegalArgumentException when an invalid status parameter is passed, resulting in an HTTP 500 Internal Server Error.

Fix & Rationale: Added a try-catch block around TaskStatus.valueOf(). Invalid values gracefully default to null (or return clean response), preventing server-side 500 crashes.

Bug 4: Negative Page Indexing SubList Crash

Location: backend/src/main/java/com/internal/tasktracker/TaskController.java (Lines 50-54)

How Discovered: Code inspection of manual subList slicing calculation.

Root Cause: If a client sends page <= 0, start = (page - 1) * pageSize evaluates to a negative number, causing subList to throw an IndexOutOfBoundsException.

Fix & Rationale: Clamped page and pageSize parameters using Math.max(1, page) so start index is never negative.

Bug 5: Pagination Reset Bug on Filter Change

Location: frontend/src/App.jsx (Lines 24-25)

How Discovered: Testing user interaction flow when changing search input or status dropdown while on later pages.

Root Cause: SearchBar and StatusFilter updated query and status states but did not reset page to 1. If on Page 3 and a filter returned only 1 page, the table showed empty results.

Fix & Rationale: Created handler functions handleQueryChange and handleStatusChange that update filter state and explicitly reset page to 1.

Bug 6: Frozen Loading State on API Failure

Location: frontend/src/hooks/useTasks.js (Lines 19-21)

How Discovered: Inspecting error handling in useTasks custom hook.

Root Cause: The .catch() callback updated the error state but forgot to call setLoading(false), leaving the component permanently stuck in "Loading tasks..." state upon network failure.

Fix & Rationale: Added setLoading(false) inside the .catch() block so the error UI renders properly.

Bug 7: Out-of-Order API Response Race Condition

Location: frontend/src/hooks/useTasks.js (Lines 10-22)

How Discovered: Analyzing React state updates during fast consecutive typing in SearchBar.

Root Cause: Rapid typing initiates multiple async fetch promises. Slower responses from earlier requests can resolve after faster responses from later requests, overwriting UI state with stale data.

Fix & Rationale: Introduced an isCurrent boolean flag inside useEffect cleanup to discard results from superseded requests.